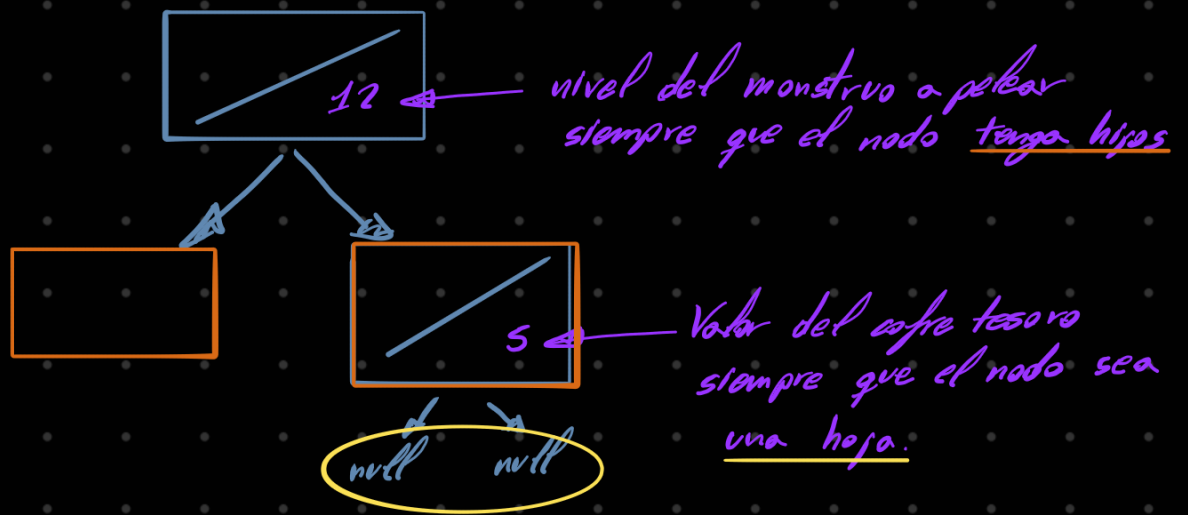
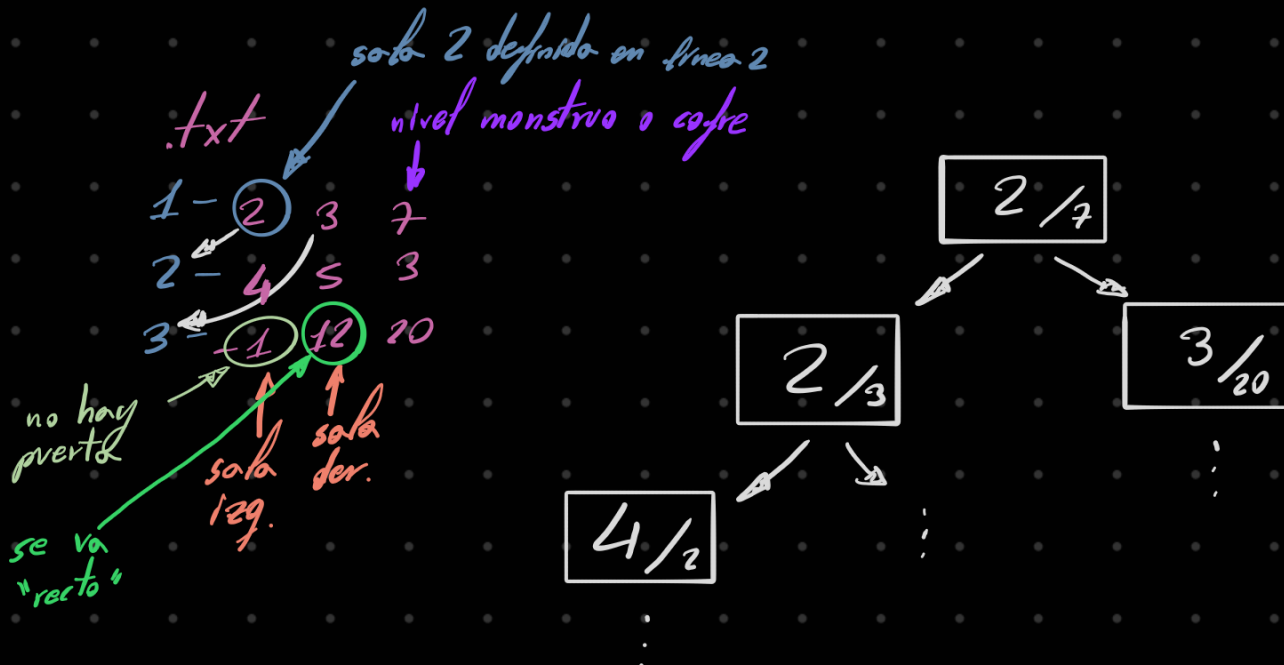




EVALUAR MAZMORRAS PROPIAS!!



Para hacer los saltos poner en private recursive poner la fila recursiva a la que hay que saltar.

StringBuilder → String como clase mutable

```
private static RetornaPuta (1) {
    int num;
    String ruta;
```

```
private static RetornaPuta getPutal() {
```

```
    return res;
```


Parte 2 ← Creamos un clase para guardar la ruta que se acaba de trazar y comparar con el tesoro más grande guardado hasta el momento.

RutaMejorTesoro()

```
if (esHoja(nodo)) {  
    // Si tiene ambos hijos nos vamos por el izq "/" y el der "\"  
    if (nodo.getIzq() != null && nodo.getDer() != null) {  
        // Datos  
        // Valor de solo  
        return Ruta(nodo.getDatos().getValor(), nodo.getDatos().getId() + " ");  
    }  
    Ruta izq = null;  
    Ruta der = null;  
    String simbolo = "";  
    if (nodo.getIzq() != null && nodo.getDer() != null) { // puede ser "/" o "\"  
        izq = rutaRekursiva(nodo.getIzq());  
        der = rutaRekursiva(nodo.getDer());  
        // llamada recursiva  
        if (der.valor > izq.valor) { // Comprobar valor ruta con mejor tesoro  
            simbolo = "\"";  
            return new Ruta(der.valor, nodo.getDatos().getId() + " " + simbolo + der.camino);  
        } else {  
            simbolo = "/";  
            return new Ruta(izq.valor, nodo.getDatos().getId() + " " + simbolo + izq.camino);  
        }  
    } else if (nodo.getIzq() != null) { // Solo puede ser "/" pero para saber el fondo a tomar  
        izq = rutaRekursiva(nodo.getIzq());  
        simbolo = "/";  
        return new Ruta(izq.valor, nodo.getDatos().getId() + " " + simbolo + izq.camino);  
    } else { // " \" por la derecha  
        der = rutaRekursiva(nodo.getDer());  
        simbolo = "\"";  
        return new Ruta(der.valor, nodo.getDatos().getId() + " " + simbolo + der.camino);  
    }  
}
```

Parte 3

```
public String rutaMasFacil() {
```

```
    String res;
```

```
    if (raiz == null) {
```

```
        res = "->";
```

```
    } else {
```

```
        Ruta mejor = buscarRutaMasFacil(raiz, 0);
```

```
        res = "->" + mejor.camino + "X";
```

```
    }
```

```
    return res;
```

```
}
```

```
private Ruta buscarRutaMasFacil(NodoArbol nodo, int sumaPrev) {
```

```
    int valorNodo = nodo.getDato().getValor();
```

```
    int nivel; Para hacer la suma acumulada
```

```
    if (valorNodo > 0) {
```

```
        nivel = valorNodo;
```

```
    } else { Si la sala es -1 lo evaluamos como 0 para que no afecte al resultado.
```

```
        nivel = 0;
```

```
    }
```

```
    int suma = sumaPrev + nivel;
```

```
    if (nodo.getIzg() == null && nodo.getDer() == null) { // es hoja
```

```
        if (valorNodo >= 0) { // Hoja con tesoro
```

```
            String hoja = nodo.getDato().getId() + "(" + valorNodo + ")";
```

```
            return new Ruta(suma, hoja);
```

```
        } else { // Hoja sin tesoro, sala vacía.
```

```
            return new Ruta(-1, "");
```

```
        }
```

```
    }
```

```
    Ruta izqRuta = null, derRuta = null;
```

```
    if (nodo.getIzg() != null) {
```

```
        izqRuta = buscarRutaMasFacil(nodo.getIzg());
```

```
    }
```

```
    if (nodo.getDer() != null) {
```

```
        derRuta = buscarRutaMasFacil(nodo.getDer());
```

```
    }
```

Minimizar suma de monstruos

Se va sumando y haciendo el recorrido al llegar a una hoja se devuelve el valor acumulado y el camino tomado en variables externas se minimiza el valor retornado de cada uno y así se guarda el camino minimizado.

// Verificamos que la ruta tenga tesoro

boolean izqValida = izqRuta != null && izqRuta.valor >= 0;

boolean derValida = derRuta != null && derRuta.valor >= 0;

Ruta mejorRuta;

String direccion;

if(izqValida && (!derValida || izqRuta.valor <= derRuta.valor)) { // izq valida y (der No o' izq valor <= der valida)

mejorRuta = izqRuta;

direccion = "/";

} else if(derValida) { // izq No valida y der si o' izq >= der valor

mejorRuta = derRuta;

direccion = "\\";

} else { // No hay tesoro

return new Ruta(-1, "");

}

String prefijo;

if(suma > 0) {

prefijo = nodo.getDato().getId() + "[" + suma + "]";

} else {

prefijo = nodo.getDato().getId() + "";

}

String camino = prefijo + direccion + mejorRuta.camino;

return new Ruta(mejorRuta.valor, camino);

}

Parte 3

Hay que hallar el camino en el que la suma del nivel sea mínimo.

Hay que como en el 2 devolver de hijos a padres varios datos

ya sea con `StringBuilder`, una estructura sencilla, una pila...

como private

poniendo en la
cima lo que me
importa.

Parte 4 — Hay que permitir claves

Aso damos el arbol de mazmorra a: monstruos y tesoros

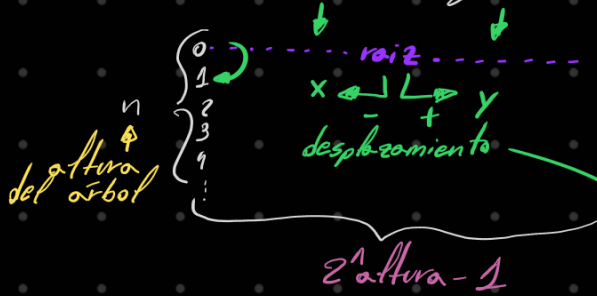
- Monstruos $\leftarrow$ if (getDer() != null && getIzq() != null && getDato() > 0)

No debería suceder pero si pasa

- Tesoros $\rightarrow$ if (`getPer == null` & & `getIzg == null` & & `getDato() == 0`) los ignoramos.

Parte 5 Fila y columna como argumento para los hijos

Se crea una matriz de strings espacios



→ El padre le dice a su hijo la
pta y columna donde tienen
que ir.

$$\text{desplazamiento} = 2^n (\text{altura} - \text{fila} - 2)$$

